Server-Side Key Management

Trusted Server Model for Zero-Effort File Sharing

CloudVault - Secure Cloud Storage System

Hybrid Encryption: AES-256-GCM + RSA-2048

The cleanest solution where the owner does absolutely NOTHING extra after upload.

1. The Key Idea

At upload, encrypt the AES key TWICE - once for the owner, and once for the server itself. Now the server can handle all sharing automatically. The owner just clicks "Share" - zero crypto work on their end.

Upload (one-time, by owner)

1. Generate random AES-256 session key
2. Encrypt file data with AES-256-GCM
3. Encrypt AES key with owner's RSA public key -> for owner's access
4. Encrypt AES key with server's RSA public key -> for server to manage sharing
5. Upload encrypted file + both encrypted keys to server

Sharing (owner just clicks "Share with User B" - that's it)

1. Server decrypts AES key using server's RSA private key
2. Server encrypts AES key with recipient's RSA public key
3. Server stores the new encrypted key entry for the recipient
4. Owner did nothing. Just one click.

Download (by any authorized user)

1. Server sends user their encrypted AES key + encrypted file
2. User decrypts AES key with their RSA private key
3. User decrypts file with AES key -> Original File

2. Owner's Experience

The owner never performs any manual cryptographic operations after the initial upload:

Action	What Owner Does	Crypto Work by Owner
Upload	Selects file, clicks Upload	Automatic (background)
Share	Selects recipient, clicks Share	None. Zero.
Revoke	Clicks Revoke	None. Server deletes key

3. Complete Flow Diagram

UPLOAD

```
Owner --> AES_Key --> Encrypt File
                |-- Encrypt with Owner's RSA Pub  --> stored (owner access)
                |-- Encrypt with Server's RSA Pub --> stored (sharing use)
```

SHARE (owner clicks one button)

```
Server --> Decrypt AES_Key with Server's RSA Priv
       --> Re-encrypt with Recipient's RSA Pub
       --> Store for recipient
       (Owner does NOTHING)
```

DOWNLOAD

```
Recipient --> Gets their RSA-encrypted AES Key
          --> Decrypts with their RSA Private Key
          --> Decrypts file with AES Key --> Original File
```

4. Why This Works Within the Project Scope

Project requirement: "secure cloud storage system using hybrid encryption (AES + RSA)... secure key management... controlled file sharing... protect from unauthorized access"

- AES + RSA hybrid - AES encrypts file data, RSA encrypts keys. Fully preserved.
- Secure key management - Server manages encrypted keys using its own RSA key pair. The AES key is NEVER stored in plaintext - it is always RSA-encrypted.
- Controlled file sharing - Owner controls WHO gets access. Server handles HOW.
- No user private key is ever shared - Each user still decrypts with their own RSA private key.
- This is exactly how real cloud systems work (AWS KMS, Google Cloud KMS, Azure Key Vault all use a trusted server key management model).

5. Security Analysis

Concern	Answer
Server sees AES key?	Yes - server is a trusted entity (standard model)
External attackers?	Protected. File=AES encrypted, keys=RSA encrypted
User private keys shared?	NEVER. Keys stay in browser (IndexedDB)
Unauthorized access?	No. Only users with a key entry can decrypt
Key stored in plaintext?	No. AES key is always RSA-wrapped

6. Implementation Architecture

Key Envelope Table (Database)

Each file has multiple key entries stored in the FileKeyEntry table. Each entry contains the AES session key encrypted with a different RSA public key:

Key Type	Encrypted With	Purpose
OWNER	Owner's RSA Public Key	Owner can download/decrypt
SERVER	Server's RSA Public Key	Server can re-encrypt for sharing
SHARED	Recipient's RSA Public Key	Recipient can download/decrypt

Technology Stack

Layer	Technology
Backend	Java 17, Spring Boot 3, Spring Security, JPA
Database	H2 (file-based, embedded)
Encryption	JCA: AES-256-GCM, RSA-2048-OAEP (SHA-256)
Frontend	React 19, Vite, Axios
Client Crypto	Web Crypto API (RSA keygen, AES decrypt)
Auth	JWT (JSON Web Tokens) + BCrypt

API Endpoints

Endpoint	Description
POST /api/auth/register	Register user (sends RSA pub key)
POST /api/auth/login	Login (returns JWT token)
POST /api/files/upload	Upload and encrypt file
GET /api/files/download/{id}	Download encrypted file
POST /api/share	Share file with another user
DELETE /api/share/{fId}/{user}	Revoke access from a user

7. Why Does the Server Store the Recipient's Key?

A common question: "Why does the server store a key entry for the recipient?" The answer: it is NOT storing the recipient's private key or public key. It stores the FILE'S AES key, encrypted FOR the recipient.

What is Actually Stored (Key Envelope Table)

Each file has multiple rows in the Key Envelope Table. Each row holds the same AES session key, but encrypted with a DIFFERENT user's RSA public key:

File ID	User	What is Stored
file_001	owner	AES key encrypted with OWNER's RSA public key
file_001	server	AES key encrypted with SERVER's RSA public key
file_001	userB	AES key encrypted with USERB's RSA public key

Why Store It?

Because User B will download the file LATER - maybe tomorrow, maybe next week. When they do:

1. Server looks up: "Does User B have a key entry for this file?"
2. YES -> sends them the encrypted AES key + encrypted file
3. User B decrypts with their RSA private key -> gets original file

If the server did NOT store it, it would have to re-encrypt every single time User B downloads. That is wasteful. Store once -> download anytime.

What is NOT Stored

- Nobody's RSA PRIVATE key - never leaves the user's browser
- The AES key in PLAINTEXT - it is always RSA-encrypted
- Only the AES key encrypted with that specific user's RSA public key (~256 bytes)

Revoke = Just Delete That Row

When the owner clicks "Revoke" for User B, the server simply DELETES that row from the Key Envelope Table. Now User B has no key entry, cannot decrypt, and has no access. The encrypted file is useless without the matching key entry.

So it is essentially an ACCESS CONTROL LIST, but cryptographically enforced. Without the key entry, the encrypted file is meaningless to anyone.

8. Conclusion

The Server-Side Key Management (Trusted Server Model) is the simplest, most practical, and zero-effort solution for the file owner. It is fully within the project's scope of AES + RSA hybrid encryption.

Key takeaways:

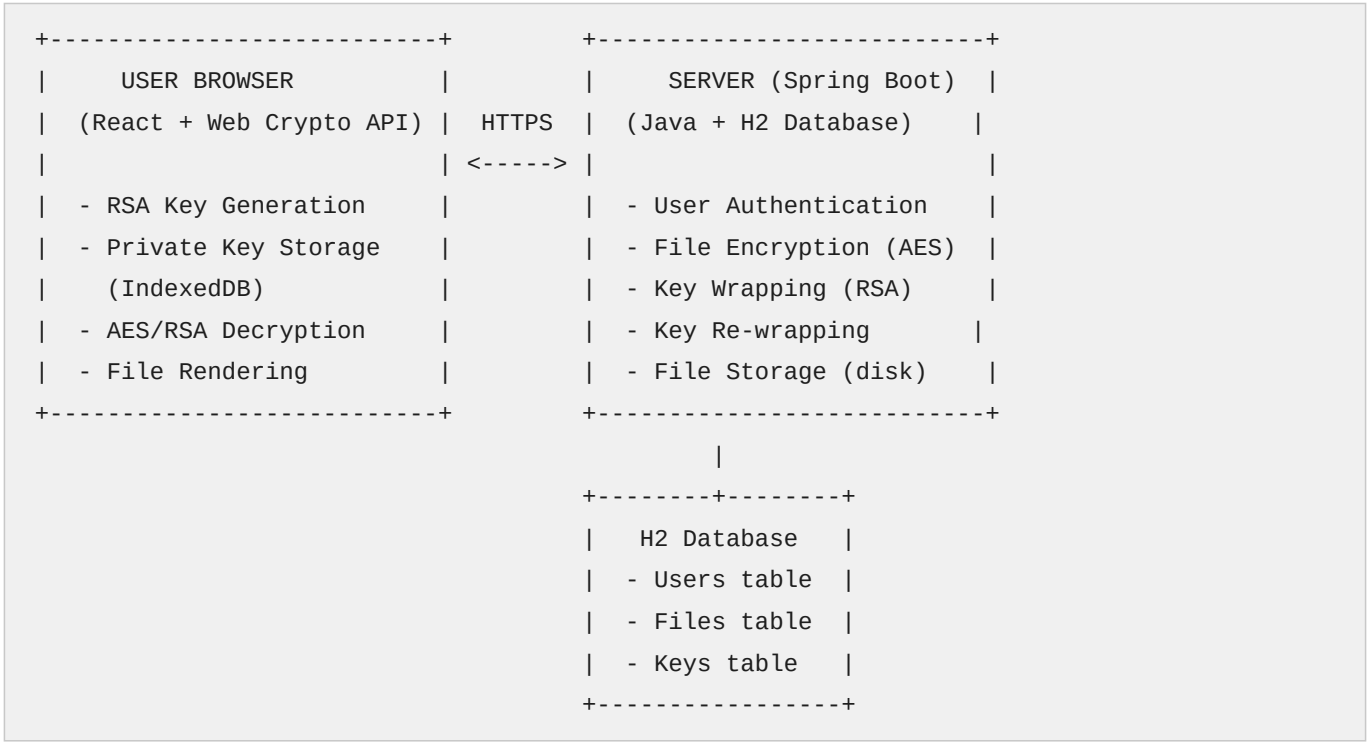
- The file is uploaded and encrypted ONCE. No re-uploads for sharing.
- The owner clicks one button to share. The server handles all re-encryption.
- Each user decrypts with their own RSA private key. No keys are ever shared.
- Revoking access is as simple as deleting the recipient's key entry.
- This model is used by AWS KMS, Google Cloud KMS, and Azure Key Vault in production.

This is the best approach that satisfies all project requirements
while providing the best possible user experience for file owners.

8. Full Working Mechanism

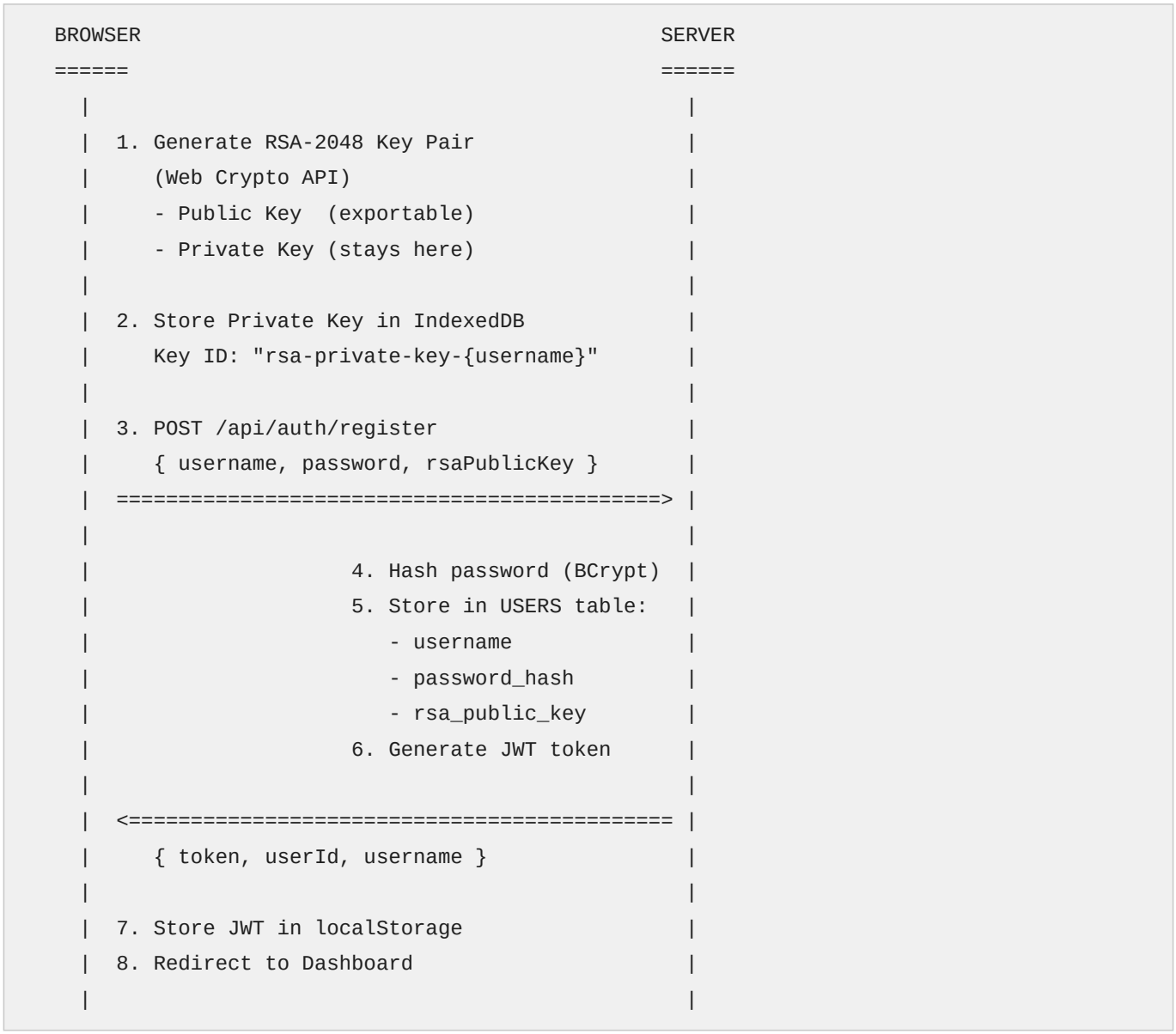
This section provides a complete end-to-end walkthrough of how CloudVault works, from user registration to file download. Each workflow is illustrated with a detailed diagram showing the exact sequence of operations, the actors involved, and where each piece of data resides at every step.

8.1 System Architecture Overview



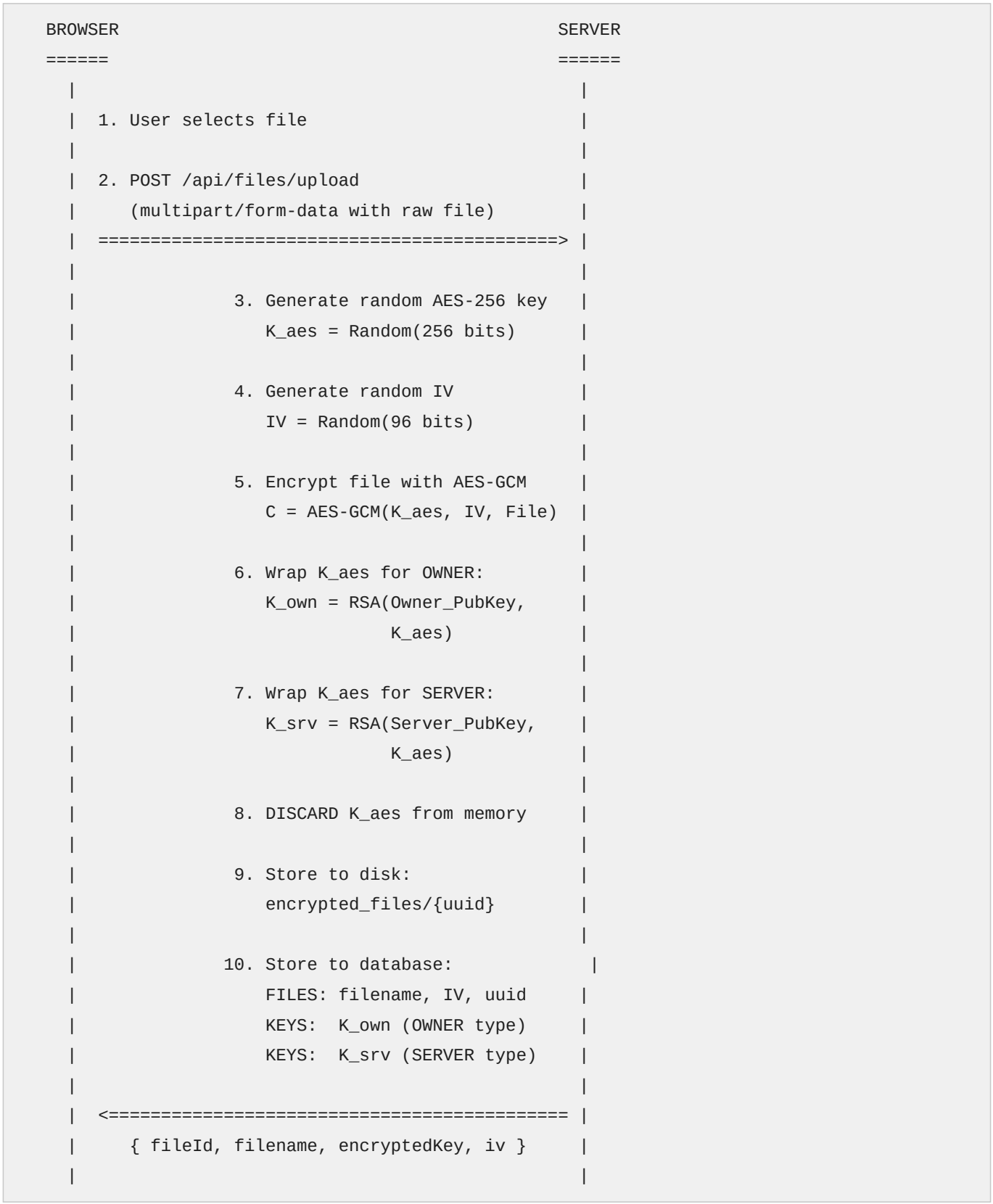
The system follows a strict separation: the browser handles RSA key pair management and final decryption, while the server handles file encryption, key wrapping, and sharing. The user's RSA private key NEVER leaves the browser.

8.2 Workflow 1: User Registration



Critical point: The private key is generated and stored ONLY in the browser's IndexedDB. It is never transmitted to the server. The server receives only the public key. Private keys are stored per-username, allowing multiple users to register on the same browser without key conflicts.

8.3 Workflow 2: File Upload & Encryption



After upload, the plaintext AES key is immediately discarded from server memory. Only two encrypted copies exist: one for the owner (decryptable by owner's private key) and one for the server (decryptable by server's private key). The original file is gone -- only the AES-GCM ciphertext remains on disk.

8.4 Workflow 3: File Sharing (Zero-Effort)

The file on disk is NOT re-encrypted, copied, or moved. Sharing only creates a new entry in the Key Envelope Table -- a 256-byte encrypted AES key. This makes sharing instant regardless of file size (a 50MB file shares just as fast as a 1KB file).

8.5 Workflow 4: File Download & Decryption

```
RECIPIENT BROWSER                                    SERVER
=================                                    ======
  |                                                    |
  |  1. Recipient clicks "Download"                    |
  |                                                    |
  |  2. GET /api/files/download/{fileId}               |
  |  ================================================> |
  |                                                    |
  |               3. Lookup SHARED key entry           |
  |                  for this user + file              |
  |                                                    |
  |               4. Read encrypted file from          |
  |                  disk (encrypted_files/{uuid})     |
  |                                                    |
  |  <================================================ |
  |     Response Body: encrypted file bytes            |
  |     Header X-Encrypted-AES-Key: K_rec              |
  |     Header X-IV: initialization vector             |
  |                                                    |
  |  5. Load private key from IndexedDB                |
  |     Key: "rsa-private-key-{username}"              |
  |                                                    |
  |  6. DECRYPT AES key with RSA:                      |
  |     K_aes = RSA_Decrypt(PrivKey, K_rec)            |
  |     (Web Crypto API - runs in browser)             |
  |                                                    |
  |  7. DECRYPT file with AES-GCM:                     |
  |     File = AES-GCM-Decrypt(K_aes, IV, C)          |
  |     (Tag verification ensures integrity)           |
  |                                                    |
  |  8. Trigger browser download                       |
  |     (original file saved to user's device)         |
  |                                                    |
```

All decryption happens CLIENT-SIDE in the browser using the Web Crypto API. The server never sees the decrypted file or the user's private key. The server only sends encrypted bytes + encrypted AES key. Even if the server is compromised, the attacker cannot decrypt files without each user's individual private key.

8.6 Complete End-to-End Flow Diagram

This diagram shows the complete lifecycle of a file from upload by Owner to download by Recipient, with all cryptographic operations:

Legend:

- PK_o = Owner's RSA Public Key, SK_r = Recipient's RSA Private Key
- PK_s = Server's RSA Public Key, SK_s = Server's RSA Private Key
- PK_r = Recipient's RSA Public Key
- K_aes = AES-256 session key, K = shorthand for K_aes

- C = AES-GCM ciphertext, IV = Initialization Vector

8.7 Key Envelope Table (Database State)

After Owner uploads a file and shares it with Recipient, the Key Envelope Table in the database looks like this:

Entry	File ID	User
OWNER	file_001	owner_id | K_own = RSA(PK_owner, K_aes)
SERVER	file_001	owner_id | K_srv = RSA(PK_server, K_aes)
SHARED	file_001	recip_id | K_rec = RSA(PK_recipient, K_aes)

Each row contains the SAME AES key but encrypted with a DIFFERENT RSA public key. Only the holder of the corresponding private key can decrypt their entry. Revoking access = deleting the SHARED row. The file remains encrypted on disk with zero changes.

9. Mathematical Foundations

The following are the pure mathematical formulas behind every cryptographic algorithm used in CloudVault.

9.1 RSA-2048 Key Generation

```
Given:
  p, q = two large random primes (~1024 bits each)

Compute:
  n      = p * q                        (RSA modulus, 2048 bits)
  phi(n) = (p - 1) * (q - 1)            (Euler's totient)
  e      = 65537                         (public exponent)
  d      = e^(-1) mod phi(n)             (private exponent)

Constraint:
  e * d  = 1  (mod phi(n))
  gcd(e, phi(n)) = 1

Output:
  Public Key  = (e, n)
  Private Key = (d, n)
```

9.2 RSA-OAEP Encryption / Decryption

```
RSA Core:
  Encrypt:  C = M^e mod n
  Decrypt:  M = C^d mod n
  Proof:    M^(e*d) mod n = M  (by Euler's theorem)

OAEP Padding (SHA-256, MGF1-SHA-256):
  r          = Random(hLen bytes)
  DB         = Hash(L) || 0x00...01 || M
  dbMask     = MGF1(r, len(DB))
  maskedDB   = DB  XOR  dbMask
  seedMask   = MGF1(maskedDB, hLen)
  maskedSeed = r  XOR  seedMask
  EM         = 0x00 || maskedSeed || maskedDB

  C = EM^e mod n      (final RSA-OAEP ciphertext)
```

9.3 AES-256-GCM (Galois/Counter Mode)

```
Inputs:
  K  = 256-bit secret key
  IV = 96-bit initialization vector
  P  = plaintext (file data)

Counter Mode Encryption:
  J_0     = IV || 0^31 || 1                  (initial counter)
  J_i     = inc_32(J_{i-1})                   (increment)
  C_i     = P_i  XOR  AES_K(J_i)              (for i = 1..n)

GHASH Authentication:
  H       = AES_K(0^128)                      (hash subkey)
  X_0     = 0^128
  X_i     = (X_{i-1}  XOR  C_i) * H           (in GF(2^128))
  X_{n+1} = (X_n  XOR  [len(A)||len(C)]) * H
  T       = X_{n+1}  XOR  AES_K(J_0)          (auth tag)

Output:
  Ciphertext = C_1 || C_2 || ... || C_n
  Tag        = T (128 bits)
```

9.4 CloudVault Hybrid Encryption Formulas

```
UPLOAD:
  K     = Random(256)                         (AES key)
  IV    = Random(96)                           (nonce)
  C,T   = AES-GCM(K, IV, File)                 (encrypt file)
  E_own = (OAEP(K))^e_owner   mod n_owner      (wrap for owner)
  E_srv = (OAEP(K))^e_server  mod n_server     (wrap for server)

SHARE:
  K     = (E_srv)^d_server    mod n_server     (unwrap server copy)
  E_rec = (OAEP(K))^e_recip   mod n_recip      (wrap for recipient)

DOWNLOAD:
  K     = (E_user)^d_user     mod n_user       (unwrap user copy)
  File  = AES-GCM-Dec(K, IV, C, T)             (decrypt + verify)
```

9.5 Key Sizes and Security Levels

Parameter	Value	Security
AES Key	256 bits (32 bytes)	128-bit quantum-safe
AES IV	96 bits (12 bytes)	NIST recommended for GCM
AES Auth Tag	128 bits (16 bytes)	Integrity guarantee
RSA Modulus	2048 bits (256 bytes)	112-bit classical
RSA Public Exp	e = 65537	Standard (Fermat F4)
OAEP Hash	SHA-256	256-bit digest
OAEP MGF	MGF1-SHA-256	Mask generation

```
Security Strength:
  AES-256:   2^256 brute-force operations required
  RSA-2048:  Factor n (~617 digit number) -- infeasible
  Combined:  Attacker must break BOTH independently
```

10. Frequently Asked Questions (FAQ)

Q1: If Person A is on Server 1 and Person B is on Server 2, how do they communicate?

In CloudVault's current design, both users are on the SAME server. It is a single centralized cloud storage system (like Google Drive or Dropbox). The server has both users' RSA public keys in its database, so sharing is just a database lookup + RSA re-encryption. No cross-server communication is needed.

If they were on different servers:

1. Server 1 requests Person B's RSA public key from Server 2 (via REST API)
2. Server 1 decrypts the AES key using its own RSA private key
3. Server 1 re-encrypts the AES key with Person B's public key
4. Server 1 sends the encrypted file + encrypted AES key to Server 2
5. Server 2 stores it for Person B -- Person B can now download and decrypt

The encryption model stays exactly the same -- AES for file, RSA for keys. The only difference is WHERE the public key comes from. On the same server, it is a database lookup. Across servers, it is an API call. The cryptography does not change. This is how real systems like Email (PGP), AWS KMS, and federated cloud systems work.

Q2: The server decrypts the AES key using its RSA private key. Can't a hacker do this too?

No. RSA has TWO keys: the Public Key (encrypts data -- anyone can do this, it is not secret) and the Private Key (decrypts data -- only the server has this). A hacker with just the public key can only ENCRYPT. They cannot reverse it.

Deriving the private key from the public key is computationally infeasible -- it would take billions of years with current technology for RSA-2048. The server's private key is stored in the keystore/ folder on the server's disk, accessible only by the server process. A hacker would need to break into the server's filesystem to steal it -- that is a server security problem (firewalls, access control, OS hardening), not an encryption design flaw.

Q3: Between RSA decryption and re-encryption, the AES key is briefly in plaintext in server memory. Is that a security gap?

Yes, the AES key exists in plaintext in server RAM for microseconds between decryption and re-encryption (literally 2 lines of Java code executing back-to-back). However, this is NOT a practical vulnerability.

Concern	Reality
How long is the gap?	Microseconds (2 lines of Java code)
Where does it exist?	Only in server RAM, never written to disk
Can a hacker read RAM?	Only with kernel-level server access
If they have server access?	They already have the private key file

To exploit this, an attacker needs kernel-level access to the server's memory. But at that point, they already have access to the server_private.key file -- the RAM gap is irrelevant. Every real cloud system has this same gap: AWS KMS (HSM memory), Google Cloud KMS, HTTPS/TLS (your bank's server), and WhatsApp (server-side message routing). This is the standard Trusted Server Model used industry-wide.

Q4: Why use HYBRID encryption (AES + RSA)? Why not just use RSA for everything?

RSA is extremely slow for large data. Encrypting a 10 MB file with RSA would take minutes and produce ciphertext many times larger than the original. RSA can only encrypt data smaller than its key size (256 bytes for RSA-2048). So encrypting a 10 MB file directly with RSA is technically impossible without breaking it into thousands of tiny blocks.

Property	AES-256-GCM	RSA-2048
Type	Symmetric (one key)	Asymmetric (key pair)
Speed	Very fast (GB/s)	Very slow (KB/s)
Max data size	Unlimited	256 bytes per block
Key sharing	Difficult (same key)	Easy (public/private)
Used for	Encrypting FILE data	Encrypting AES KEY

Hybrid encryption gives us the BEST of both: AES speed for bulk data encryption, and RSA key-pair convenience for secure key exchange. The AES key is only 32 bytes -- perfectly sized for RSA encryption. This is the same approach used by TLS/HTTPS, SSH, PGP, S/MIME, and every modern secure communication protocol.

Q5: Why AES-GCM instead of AES-CBC? What is the difference?

AES-GCM (Galois/Counter Mode) provides AUTHENTICATED encryption -- it guarantees both confidentiality AND integrity. AES-CBC (Cipher Block Chaining) provides only confidentiality.

Feature	AES-GCM	AES-CBC
Confidentiality	Yes	Yes
Integrity check	Yes (auth tag)	No
Tamper detection	Yes (fails if modified)	No (silent corruption)
Padding needed	No (stream cipher)	Yes (PKCS5/7)
Padding oracle attack	Not vulnerable	Vulnerable
Performance	Faster (parallelizable)	Slower (sequential)

With AES-CBC, an attacker could modify the encrypted file without detection -- the decrypted output would be corrupted but the system would not know. With AES-GCM, any modification is detected because the 128-bit authentication tag will fail verification. AES-CBC is also vulnerable to padding oracle attacks (like the famous POODLE attack). AES-GCM has no padding, so this entire class of attacks is eliminated. This is why modern standards (TLS 1.3, NIST) recommend GCM over CBC.

Q6: Why RSA-2048? Is it secure enough? Why not RSA-4096?

RSA-2048 provides approximately 112 bits of security. NIST considers this secure until at least 2030. No one has ever factored a 2048-bit RSA key, and it would take classical computers billions of years to do so.

RSA Key Size	Security Level	Status
1024 bits	80 bits	BROKEN -- do not use
2048 bits	112 bits	Secure until 2030+ (NIST)
3072 bits	128 bits	Secure until 2040+
4096 bits	140+ bits	Overkill for most uses

RSA-4096 is 4-8x slower than RSA-2048 for key generation, encryption, and decryption. For a cloud storage project, this slowdown impacts user experience (registration, sharing) with negligible security gain. RSA-2048 is the industry standard used by most certificate authorities, banks, and government systems today. Our project uses RSA-2048 with OAEP padding (SHA-256), which is the recommended configuration by NIST SP 800-56B.

Q7: When you revoke access, the user already downloaded the file. How is revocation truly effective?

This is an important and honest limitation. Revocation in CloudVault works as follows:

What revocation DOES:

- Deletes the recipient's encrypted AES key entry from the Key Envelope Table
- Prevents any FUTURE downloads -- the server will refuse the request
- Even if the user has the encrypted file cached, they cannot get the AES key again

What revocation CANNOT do:

- Cannot delete a file the user already downloaded and saved to their local disk
- Cannot erase the decrypted content from the user's device

This is the same limitation in EVERY cloud sharing system: Google Drive, Dropbox, OneDrive, and even DRM systems. Once a user has decrypted and saved data locally, no server-side action can undo that. The solution is the same as in enterprise systems: legal agreements (NDAs), audit logs, and trust. Revocation prevents future access, not past access. This is a fundamental computer science limitation, not a design flaw.

Q8: Can a Man-in-the-Middle (MITM) attacker intercept the file during download?

Even if an attacker intercepts the download, they get ONLY encrypted data. The intercepted data consists of two parts:

1. The encrypted file (AES-256-GCM ciphertext) -- useless without the AES key
2. The AES key encrypted with the USER's RSA public key -- useless without their private key

The attacker would need the user's RSA private key to decrypt the AES key, and then use that AES key to decrypt the file. But the user's private key NEVER leaves their browser (stored in IndexedDB via Web Crypto API). It is never transmitted over the network.

Additionally, in a production deployment, all communication uses HTTPS (TLS), which adds another layer of encryption to the transport itself. So the attacker would need to break TLS first, then break RSA-2048, then break AES-256 -- three independent layers. This defense-in-depth approach makes MITM attacks practically impossible.

Q9: What happens if the user loses their RSA private key (clears browser data)?

If the user clears their browser data (IndexedDB), their RSA private key is permanently lost. The consequences are:

- The user can NO LONGER decrypt any files shared with them or their own files
- The encrypted files still exist on the server but are unrecoverable by that user
- The server CANNOT help because it does not have the user's private key

Recovery options in our system:

1. For OWNED files: The server has its own copy of the AES key (SERVER entry). The user re-registers (new key pair), and the server can re-encrypt the AES key with the new public key -- restoring access.
2. For SHARED files: The owner can simply re-share the file with the user after they re-register with a new key pair.

This is actually a design advantage of the Trusted Server Model over pure End-to-End encryption. In E2E systems (like Signal), losing your private key means ALL data is permanently lost with no recovery path. In our system, the server's copy of the AES key acts as a recovery mechanism while still maintaining encryption at rest.

Production systems solve this with:

- Key escrow -- encrypted backup of private key (e.g., protected by master password)
- Multi-device sync -- private key synced across devices (like iCloud Keychain)
- Recovery keys -- one-time recovery codes generated at registration

Q10: Will quantum computers break this encryption? Is the system future-proof?

Quantum computers pose a theoretical threat to RSA but NOT to AES:

Algorithm	Quantum Threat	Timeline
RSA-2048	BROKEN by Shor's algorithm	10-20+ years away
AES-256	WEAKENED to 128-bit (Grover)	Still secure
AES-128	WEAKENED to 64-bit (Grover)	Potentially broken

Shor's algorithm can theoretically factor RSA keys in polynomial time. However, a quantum computer capable of breaking RSA-2048 would need approximately 4,000 logical qubits. As of 2024, the largest quantum computers have around 1,000 noisy physical qubits -- far from the millions of error-corrected qubits needed. Most experts estimate this is 10-20+ years away.

AES-256 is quantum-resistant. Grover's algorithm reduces the effective key strength by half (256 bits becomes 128 bits), but 128-bit security is still considered unbreakable. This is why we chose AES-256 specifically.

Post-Quantum migration path:

When quantum computers become a real threat, the RSA component can be replaced with post-quantum algorithms like CRYSTALS-Kyber (NIST standardized in 2024 as ML-KEM) or CRYSTALS-Dilithium for signatures. The AES-256-GCM layer remains unchanged. This modular hybrid design makes the migration straightforward -- only the key wrapping layer needs to be swapped, not the entire encryption pipeline.

Q11: When does each cryptographic operation actually happen -- on Share click or on Download click?

This is a crucial architectural question. The operations are split across two completely separate moments in time:

When Owner clicks "Share" (Server-side, instant):

1. Server decrypts AES key using Server's RSA Private Key
2. Server encrypts AES key with Recipient's RSA Public Key
3. Server stores the new encrypted key entry in the database

IMPORTANT: The actual encrypted FILE is NEVER touched during sharing. Only the tiny 32-byte AES key is re-wrapped. The file remains exactly as-is on disk. This is why sharing is instant regardless of file size.

When Recipient clicks "Download" (Client-side, in browser):

4. Server sends the encrypted file + recipient's encrypted AES key
5. Browser decrypts AES key using Recipient's RSA Private Key (from IndexedDB)
6. Browser decrypts the file using the recovered AES key + IV
7. Decrypted original file is saved to user's device

Operation	When?	Where?
RSA decrypt (server key)	Share click	Server (Java backend)
RSA encrypt (recipient key)	Share click	Server (Java backend)
RSA decrypt (user key)	Download click	Browser (Web Crypto API)
AES-GCM decrypt file	Download click	Browser (Web Crypto API)

This split design is intentional: sharing is a lightweight server operation (only re-wrapping a 32-byte key), while the heavy decryption work happens in the recipient's browser. The server never decrypts the actual file content -- it only handles key management. This separation of concerns is what makes the system both secure and efficient.

These FAQs cover the most critical security and design questions
that demonstrate deep understanding of the CloudVault architecture.